

```
int board[51][51];
```

메두사 구조체

0에서 $N - 1$ 의 범위로 이루어진 $N \times N$ 크기의 어느 마을에 메두사가 살고 있습니다. 이 마을에는 도로가 깔려 있으며, 도로는 0, 도로가 아닌 곳은 1로 주어집니다.

메두사는 집에서 공원까지 산책을 나가기로 했습니다. 메두사의 집은 좌표 (S_r, S_c) 에 있고, 공원은 좌표 (E_r, E_c) 에 위치해 있습니다. 메두사는 오직 도로만을 따라 최단 경로로 공원까지 이동합니다. 이때 메두사의 집과 공원은 항상 도로위에 있으며, 집과 공원의 좌표는 다름을 가정해도 좋습니다.

전사 구조체

```
Warrior warriors[301];
```

그런데 오늘은 M 명의 용감한 전사들이 메두사를 잡기 위해 마을에 도착했습니다. 각 전사는 초기에 (r_i, c_i) 에 위치해있으며, 메두사를 향해 최단 경로로 이동합니다. 전사들은 도로와 비도로를 구분하지 않고 어느 칸이든 이동할 수 있습니다. 메두사의 집에 전사들이 초기부터 위치하는 경우 또한 없음을 가정해도 좋습니다.

전사들이 마을에 들어와있음을 미리 알고 있던 메두사는 전사들이 움직이기 전에 그들을 바라봄으로써 도로 만들어 움직임을 멈출 수 있습니다.

메두사와 전사들의 이동은 매턴마다 다음 순서에 맞춰 차례로 진행됩니다.

[1] 메두사의 이동 1번 동작

경로 복원 알고리즘 → 최단 경로 구해놓고 따라가기.

메두사는 도로를 따라 한 칸 이동하며, 공원까지 최단 경로를 따릅니다. 메두사가 이동한 칸에 전사가 있을 경우 전사는 메두사에게 공격을 받고 사라집니다. 만약 집으로부터 공원까지 여러 최단경로가 가능하다면 상, 하, 좌, 우의 우선순위를 따릅니다. 또한 메두사의 집으로부터 공원까지 도달하는 경로가 없을 수도 있음을 유의하세요.

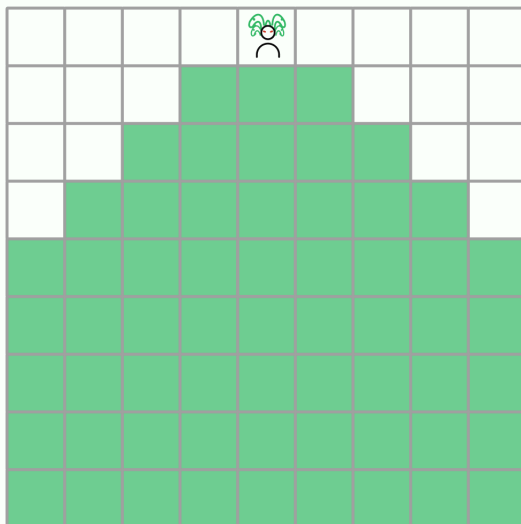
-1 출력

[2] 메두사의 시선 2번 동작

메두사는 상, 하, 좌, 우 하나의 방향을 선택해 바라봅니다.

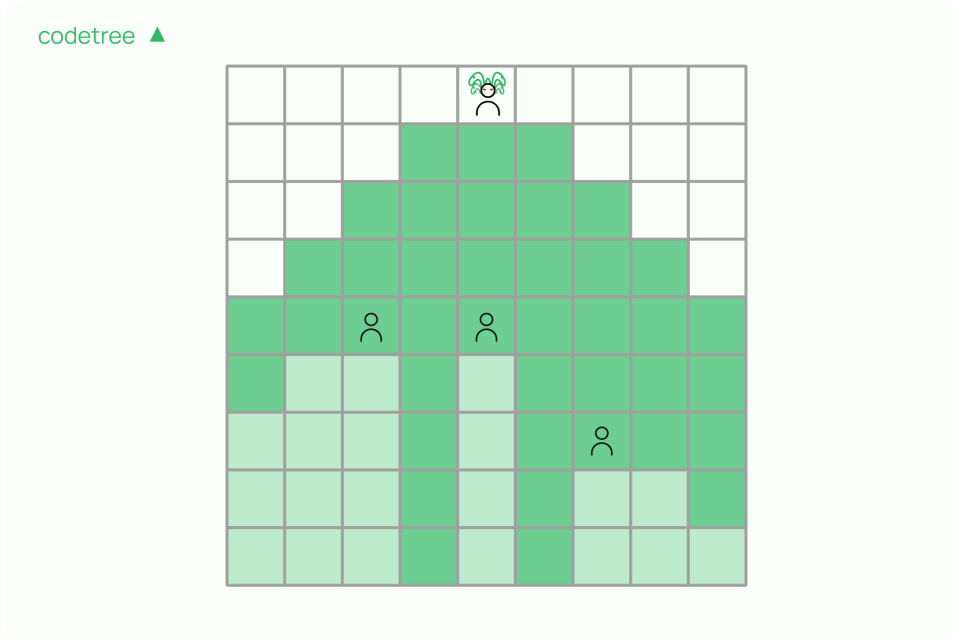
메두사는 바라보는 방향으로 90도의 시야각을 가지며, 시야각 범위 안에 있는 전사들을 볼 수 있습니다. 아래 그림은 메두사가 아래 방향을 바라볼 때 시야를 나타냅니다.

codetree ▲



메두사의 시야각 안에 들어와있지만 다른 전사에 가려진 전사의 경우 메두사에게 보이지 않습니다.

특정 전사에 의해 메두사에게 가려지는 범위는 메두사와 해당 전사의 상대적인 위치에 의해 결정됩니다. 가려지는 칸들은 메두사가 바라보는 방향을 따라 그 전사의 뒤편으로 퍼집니다. 전사가 메두사와 같은 행 또는 열(직선 방향)에 있다면 그 전사의 뒤로 곧게 뻗은 한 줄이 가려지고, 전사가 대각 방향에 있다면 메두사가 바라보는 방향과 그 대각 방향 사이를 채우며 삼각형 모양으로 점점 넓어지는 영역이 가려집니다.



전사 구조체에 isStone 변수.

이번 턴 이후에 바로 풀려나니까 isStone은 bool type.

메두사가 본 전사들은 모두 돌로 변해 현재 턴에는 움직일 수 없으며, 이번 턴이 종료되었을 때 돌에서 풀려납니다. 만약 두 명 이상의 전사들이 같은 칸에 위치해있다면 해당 칸의 전사들은 모두 돌로 변하게 됩니다.

전사들 중첩 가능

이때 메두사는 상,하,좌,우 중 전사를 가장 많이 볼 수 있는 방향을 바라봅니다. 같은 수의 전사를 볼 수 있는 방향이 여러 개라면, 상하좌우의 우선순위로 방향을 결정합니다.

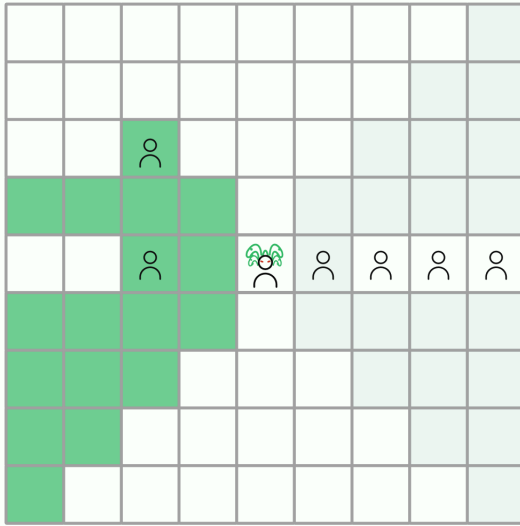
아래 그림과 같은 상황에서는 좌측을 바라볼 때 두 명의 전사, 우측을 바라볼 때 한 명의 전사를 볼 수 있으므로 좌측을 바라보게 됩니다.

다행히도 메두사의 이동 방향 우선 순위 = 메두사의 시선 방향 우선 순위

메두사의 시선은 방향도 정하고, 쳐다보고, 시야 가려지고, 돌로 변하는 복잡한 과정.

↳ 분리.

ex. 방향 정하는 동작, 쳐다보는 영역 구하는 동작, 돌로 변하는 동작.



[3] 전사들의 이동 3번 동작

돌로 변하지 않은 전사들은 메두사를 향해 최대 두 칸까지 이동합니다. 전사들은 이동 중 같은 칸을 공유할 수 있습니다.

- 첫 번째 이동
 - 메두사와의 거리를 줄일 수 있는 방향으로 한 칸 이동합니다. 이런 방향이 두 개 이상일 경우 상하좌우의 우선순위로 방향을 선택합니다. *여기까지도 dx/dy 우선 순위 동일.*
 - 격자의 바깥으로 나갈 수 없으며, 메두사의 시야에 들어오는 곳으로는 이동할 수 없습니다.
- 두 번째 이동
 - 메두사와의 거리를 줄일 수 있는 방향으로 한 칸 더 이동합니다. 이런 방향이 두 개 이상일 경우 좌우상하의 우선순위로 방향을 선택합니다. $(d+2) \% 4$
 - 격자의 바깥으로 나갈 수 없으며, 메두사의 시야에 들어오는 곳으로는 이동할 수 없습니다.

[4] 전사의 공격 4번 동작

메두사와 같은 칸에 도달한 전사는 메두사를 공격합니다. 그러나 메두사가 너무 강한 나머지 이기지 못하고 사라집니다. 전사 구조체에 *isDead 변수 (bool)*

위의 네 단계에서 최단경로를 계산할 때는 맨해튼 거리를 기준으로 합니다.

위의 네 단계가 반복되어 메두사가 공원에 도달할 때까지 ^{while} 매 턴마다 해당 턴에서 모든 전사가 이동한 거리의 합, 메두사로 인해 돌이 된 전사의 수, 메두사를 공격한 전사의 수 *출력*를 공백을 사이에 두고 차례대로 출력하는 프로그램을 작성하세요. 단, 메두사가 공원에 도착하는 턴에는 0을 출력하고 프로그램을 종료합니다.

입력

제한 조건

- $4 \leq N \leq 50$
- $0 \leq M \leq 300$
- $0 \leq s_r, s_c, e_r, e_c, a_{ir}, a_{ic} \leq N - 1$

출력

입력 예제

예제 1

입력

```
4 4
1 3 3 3
3 1 0 3 1 0 2 2
0 0 0 0
0 0 0 0
0 0 1 1
1 0 0 0
```

출력

```
4 2 2
0 2 0
1 1 1
0 0 0
0 0 0
0
```

예제 2

입력

```
6 4
3 1 1 2
3 5 1 4 0 4 1 3
0 0 0 0 1 0
0 1 0 0 1 1
1 1 0 0 0 0
0 0 1 0 1 1
```

```
if (FindMedusaRoad()) MakeMedusaRoad();
else {
    cout << -1;
    return;
}
while (1) {
    totalD = stoneCnt = attackerCnt = 0;
    MoveMedusa();
    LookMedusa();
    MakeStone();
    MoveWarriors();
    ResetStone();
    cout << totalD << ' ' << ... << '\n';
}
cout << 0;
```

```
0 0 0 0 0 0
0 0 0 0 1 1
```

출력

```
3 2 0
2 2 1
0 2 0
0 2 0
0 2 0
1 1 1
0
```

제한